2026

MODUL PRAKTIKUM 2

AUDIO FEATURE EXTRACTIONS

OPTIMIZED VERSION

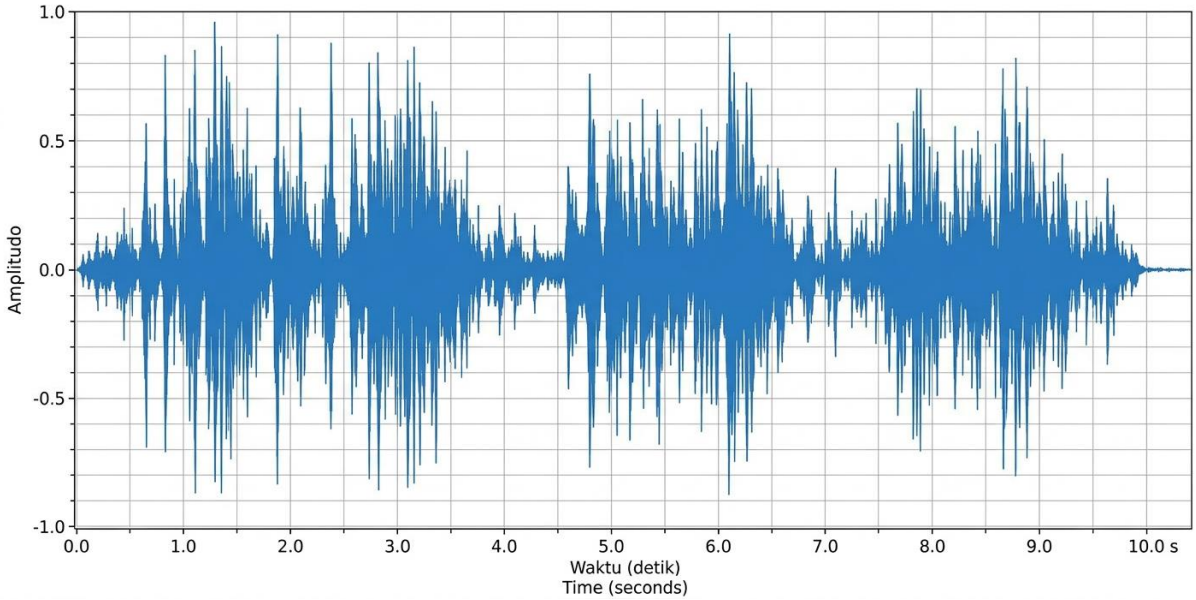
Disusun Oleh:
Ryan F F Hakim

Dipersembahkan Untuk:
Sains Data

Mata Kuliah
Sistem Multimedia

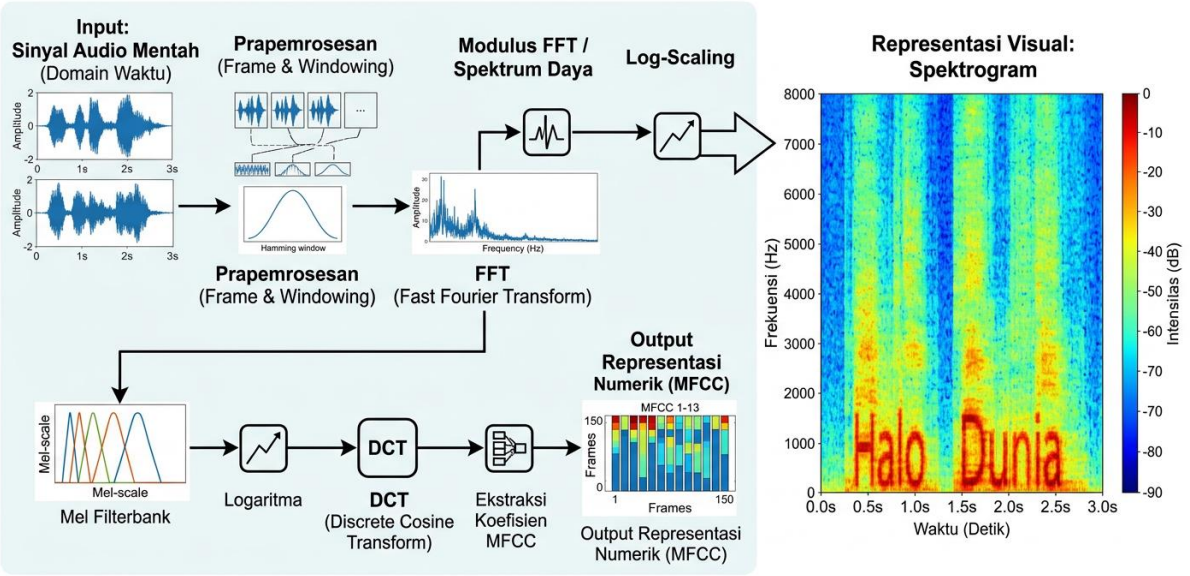
PENDAHULUAN

Audio merupakan salah satu komponen paling fundamental dan esensial dalam ekosistem sistem multimedia modern. Bersama dengan teks, gambar, dan video, sinyal audio memperkaya interaksi manusia dengan teknologi, memberikan dimensi audial yang vital bagi penyampaian informasi. Seiring dengan pesatnya perkembangan teknologi digital, volume data suara yang dihasilkan dan dipertukarkan setiap harinya semakin masif. Namun, data audio mentah (*raw audio*) pada dasarnya hanyalah deretan fluktuasi tekanan udara yang direkam dalam bentuk sinyal digital yang sangat kompleks dan berdimensi tinggi. Hal ini menghadirkan tantangan tersendiri ketika sistem komputer diinstruksikan untuk memproses, memahami, dan mengekstrak makna dari suara tersebut secara otomatis.



Gambar 1.1: Representasi sinyal audio mentah (raw audio) dalam bentuk gelombang suara pada ranah waktu.

Untuk mengatasi kompleksitas dan besarnya dimensi data tersebut, diperlukan sebuah tahapan komputasional krusial yang dikenal sebagai ekstraksi fitur (*feature extraction*). Ekstraksi fitur adalah proses transformasi matematis yang bertujuan untuk mengubah sinyal audio mentah menjadi representasi numerik yang jauh lebih ringkas, padat, dan informatif. Melalui tahapan ini, berbagai informasi yang redundan atau tidak relevan (seperti derau/ *noise*) dapat direduksi, sementara karakteristik unik dari suara—seperti frekuensi, amplitudo, timbre, hingga ritme—ditangkap dan dikuantifikasi ke dalam parameter-parameter terukur. Representasi ini sering kali direpresentasikan dalam bentuk vektor atau matriks fitur.



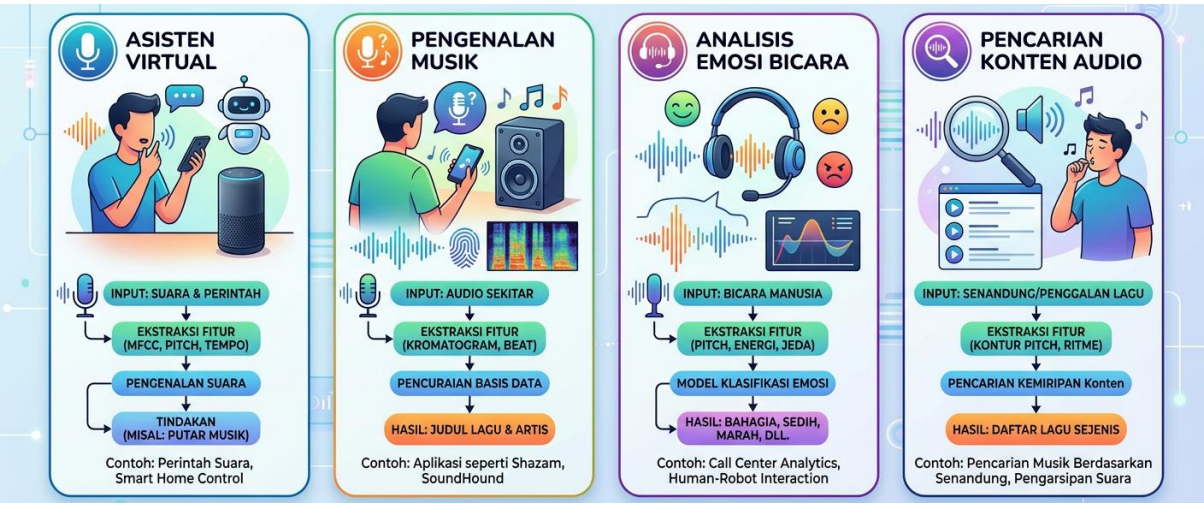
Gambar 1.2: Proses transformasi sinyal audio mentah menjadi representasi visual dan numerik seperti Spektrogram atau MFCC.

Keberhasilan sebuah sistem multimedia cerdas sangat bergantung pada kualitas fitur yang diekstrak. Representasi numerik hasil ekstraksi ini berfungsi sebagai fondasi utama bagi algoritma pembelajaran mesin (*machine learning*), jaringan saraf tiruan (*deep learning*), dan sistem kecerdasan buatan lainnya. Dengan input data yang telah disederhanakan namun tetap memuat esensi dari sinyal aslinya, model komputasi dapat berlatih untuk mengenali pola, membedakan jenis suara, dan melakukan analisis prediktif dengan tingkat akurasi serta efisiensi komputasi yang tinggi.

Saat ini, pemanfaatan fitur audio telah diimplementasikan secara luas dan menjadi tulang punggung bagi berbagai inovasi aplikasi multimedia mutakhir di berbagai industri. Beberapa implementasi utamanya meliputi:

- Pengenalan Ucapan (*Speech Recognition*): Teknologi yang memungkinkan mesin untuk menyalin teks dari suara manusia secara aktual (*real-time*), yang menjadi dasar bagi sistem dikte otomatis dan asisten virtual pintar.
- Pencarian dan Klasifikasi Musik (*Music Information Retrieval / MIR*): Bidang yang berfokus pada analisis struktur lagu untuk memfasilitasi pengelompokan berdasarkan genre, sistem rekomendasi musik (seperti pada platform *streaming*), hingga identifikasi lagu dari cuplikan melodi.
- Deteksi Kejadian Berbasis Suara (*Audio Event Detection*): Pemanfaatan analisis suara dalam sistem pengawasan dan keamanan pintar untuk mendeteksi anomali lingkungan, seperti suara kaca pecah, ledakan, atau tangisan bayi.

- Identifikasi Pembicara (*Speaker Identification*): Sistem otentikasi biometrik yang bertugas mengenali dan memverifikasi identitas individu secara spesifik berdasarkan karakteristik dan profil vokal yang unik.
- Pengenalan Emosi (*Emotion Recognition*): Teknologi analisis suara tingkat lanjut yang membantu sistem mendeteksi kondisi psikologis, sentimen, atau suasana hati pengguna melalui analisis intonasi, tempo, dan nada bicara.



Gambar 1.3: Berbagai implementasi sistem ekstraksi fitur audio pada aplikasi multimedia cerdas di dunia nyata.

Melalui modul praktikum ini, praktikan akan diajak untuk memahami serta mempraktikkan secara langsung tahapan pemrosesan sinyal audio dari hulu ke hilir. Diharapkan setelah menyelesaikan modul ini, mahasiswa mampu mengimplementasikan berbagai metode ekstraksi fitur audio standar dan memahami perannya dalam membangun sistem multimedia cerdas yang aplikatif di dunia nyata.

DEPENDENSI

CODE

```
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
from IPython.display import Audio
import os
```

CODE

```
# Memastikan direktori hasil ada
os.makedirs("results", exist_ok=True)
```

DASAR-DASAR AUDIO DIGITAL

2.1 Karakteristik Dasar Audio Digital

Suara pada dasarnya adalah gelombang analog berupa fluktuasi tekanan udara yang bergerak secara kontinu. Agar dapat diproses oleh komputer, gelombang analog ini harus didigitalisasi melalui proses **Sampling** dan **Kuantisasi**.

Beberapa parameter krusial dalam audio digital antara lain:

- **Frekuensi Sampling (*Sampling Rate*):** Jumlah sampel atau titik data yang diambil dari sinyal analog dalam satu detik. Satuannya adalah Hertz (Hz). Berdasarkan Teorema Nyquist-Shannon, *sampling rate* harus minimal dua kali lipat dari frekuensi tertinggi yang ingin direkam agar tidak terjadi distorsi (*aliasing*).
- **Jumlah Sampel (*Total Samples*):** Total keseluruhan titik data yang merepresentasikan keseluruhan file audio.
- **Durasi Audio:** Durasi waktu sinyal dapat dihitung menggunakan hubungan antara jumlah sampel (NN) dan *sampling rate* (f_s):

$$T = \frac{N}{f_s}$$

Di mana T adalah durasi dalam satuan detik.

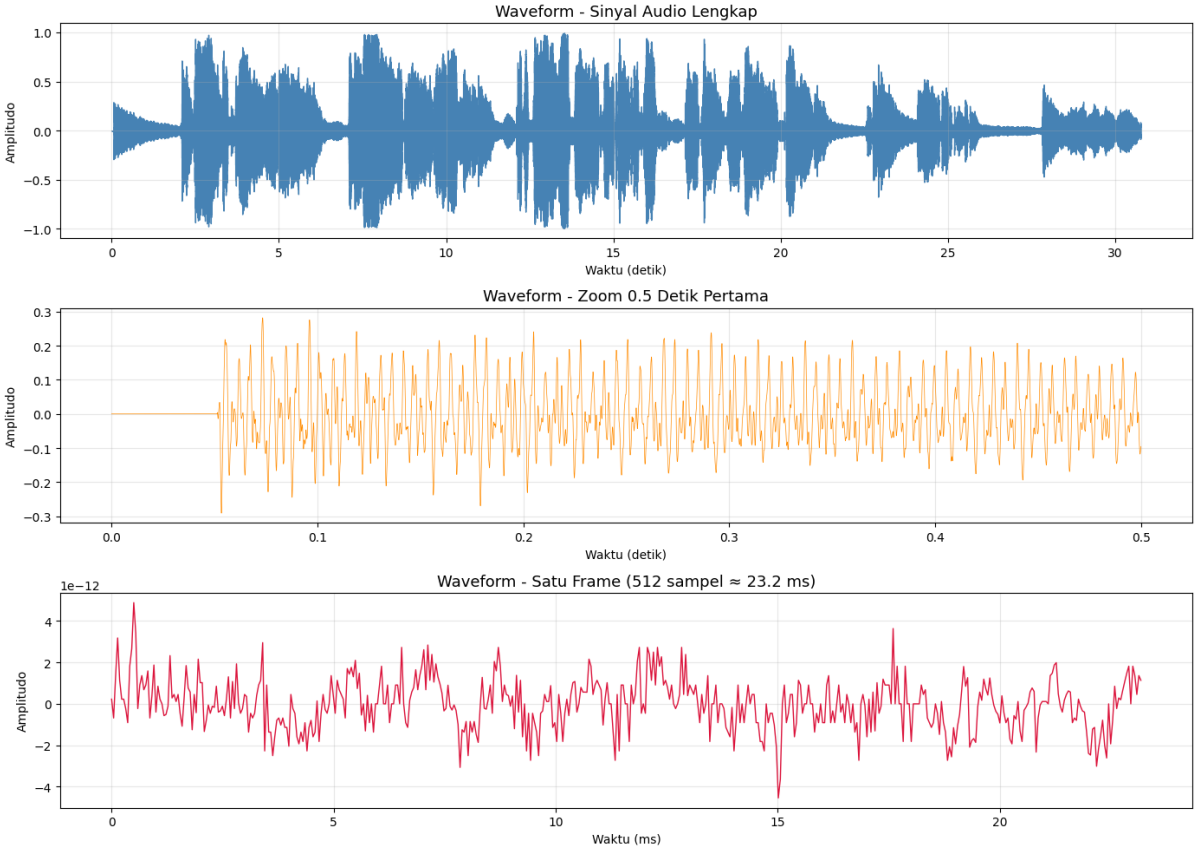
CODE

```
# 1. Memuat file audio

file_path = "/content/Echoes_in_the_Dust.mp3"

# Memuat audio dengan librosa
# sr=None untuk mempertahankan sampling rate asli
# sr=22050 untuk resampling ke 22050 Hz (default librosa)
y, sr = librosa.load(file_path, sr=22050)

# Informasi dasar audio
print(f"Sampling Rate      : {sr} Hz")
print(f"Jumlah Sampel      : {len(y)}")
print(f"Durasi             : {len(y)/sr:.2f} detik")
print(f"Tipe Data          : {y.dtype}")
print(f"Nilai Min          : {y.min():.4f}")
print(f"Nilai Max          : {y.max():.4f}")
print()

# Memutar audio di Jupyter Notebook
Audio(y, rate=sr)
```

2.2 Representasi Sinyal pada Ranah Waktu (*Waveform*)

Bentuk paling dasar dari representasi audio digital adalah *Waveform* (gelombang suara). *Waveform* menampilkan sinyal dalam **ranah waktu** (*time domain*), di mana sumbu horizontal merepresentasikan waktu (detik atau milidetik) dan sumbu vertikal merepresentasikan amplitudo (intensitas sinyal).

Analisis *waveform* berguna untuk melihat amplop energi suara (*energy envelope*), mengidentifikasi bagian yang sunyi (*silence*), dan melihat bentuk gelombang secara makro. Namun, informasi pada ranah waktu saja tidak cukup untuk membedakan nada, timbre, atau fonem karena frekuensi penyusunnya saling bertumpuk dan tidak terlihat secara eksplisit.



Gambar 2.1: Visualisasi Waveform

CODE

```
# 2. Visualisasi Waveform

fig, axes = plt.subplots(3, 1, figsize=(14, 10))

# --- Plot 1: Waveform lengkap ---
axes[0].set_title("Waveform - Sinyal Audio Lengkap", fontsize=13)
librosa.display.waveshow(y, sr=sr, ax=axes[0], color='steelblue')
axes[0].set_xlabel("Waktu (detik)")
axes[0].set_ylabel("Amplitudo")
axes[0].grid(True, alpha=0.3)

# --- Plot 2: Zoom ke 0.5 detik pertama ---
samples_05 = int(0.5 * sr)
time_axis = np.arange(samples_05) / sr

axes[1].set_title("Waveform - Zoom 0.5 Detik Pertama", fontsize=13)
axes[1].plot(time_axis, y[:samples_05], color='darkorange',
linewidth=0.5)
axes[1].set_xlabel("Waktu (detik)")
axes[1].set_ylabel("Amplitudo")
axes[1].grid(True, alpha=0.3)

# --- Plot 3: Zoom ke 1 frame (\~23ms = 512 sampel) ---
frame_size = 512
time_frame = np.arange(frame_size) / sr

axes[2].set_title(f"Waveform - Satu Frame ({frame_size} sampel ≈
{frame_size/sr*1000:.1f} ms)", fontsize=13)
axes[2].plot(time_frame * 1000, y[:frame_size], color='crimson',
linewidth=1)
axes[2].set_xlabel("Waktu (ms)")
axes[2].set_ylabel("Amplitudo")
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("results/01_waveform.png", dpi=150, bbox_inches='tight')
plt.show()
```

2.3 Framing dan Windowing

Sinyal audio, terutama suara manusia dan musik, bersifat **non-stasioner**, artinya karakteristik frekuensinya berubah-ubah seiring waktu. Jika kita menganalisis seluruh sinyal secara sekaligus, kita akan kehilangan informasi kapan sebuah frekuensi muncul. Oleh karena itu, sinyal harus dipecah menjadi segmen-segmen kecil yang disebut **Frame**.

- **Framing:** Memotong sinyal menjadi bingkai-bingkai pendek (umumnya berdurasi 20–40 milidetik). Dalam durasi sesingkat ini, sinyal diasumsikan bersifat pseudo-stasioner (karakteristiknya tidak banyak berubah).
- **Overlap (Tumpang Tindih):** Untuk mencegah hilangnya informasi di batas pergantian frame, ekstraksi dilakukan dengan pergeseran (*hop length*) yang lebih kecil dari ukuran frame, sehingga antar frame terjadi tumpang tindih.

Windowing Proses pemotongan sinyal secara tiba-tiba (membentuk kotak/ *rectangular*) akan menghasilkan patahan atau diskontinuitas di ujung frame. Secara matematis, hal ini memunculkan frekuensi palsu yang disebut kebocoran spektral (*spectral leakage*). Untuk menghaluskannya, setiap frame dikalikan dengan fungsi jendela (*window function*) yang nilainya mengecil di kedua ujungnya.

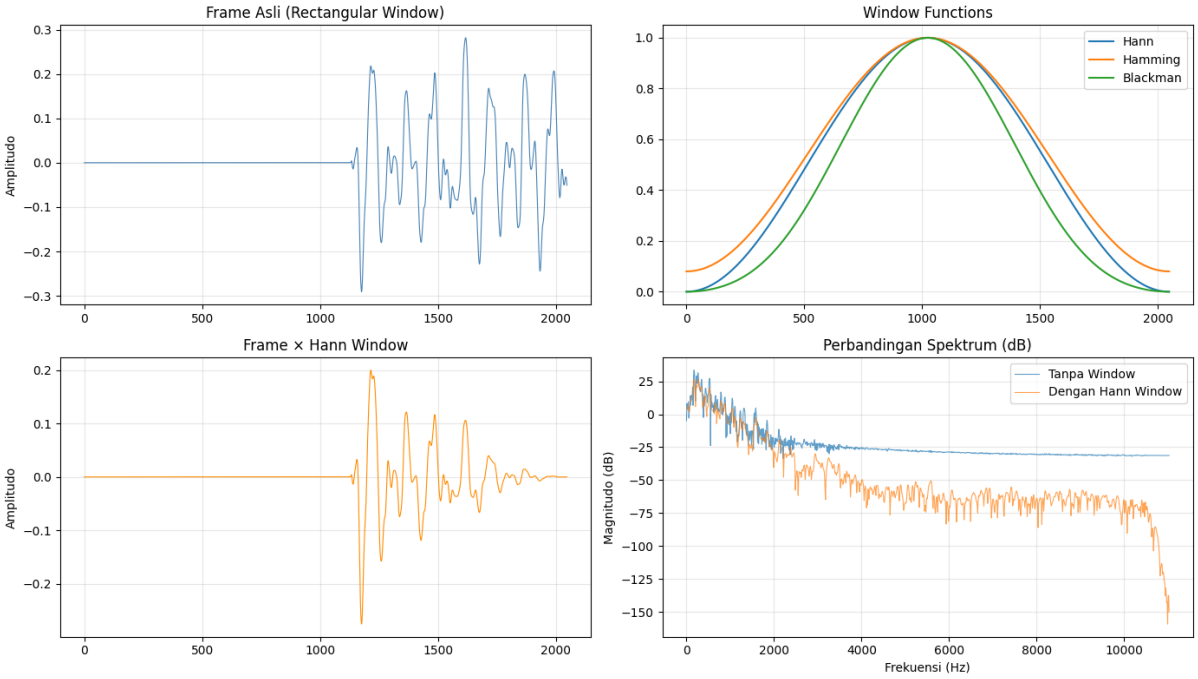
Persamaan umum windowing adalah:

$$x_w[n] = x[n] \cdot w[n]$$

Di mana $x_w[n]$ adalah sinyal setelah di-window, $x[n]$ adalah sinyal asli pada frame tersebut, dan $w[n]$ adalah fungsi jendela. Salah satu fungsi jendela yang paling umum digunakan adalah **Jendela Hann** (*Hann Window*), dengan persamaan:

$$w[n] = 0.5\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right)$$

Di mana N adalah panjang frame dan nn adalah indeks sampel ($0 \leq n \leq N-1$).



Gambar 2.2: Framing dan Windowing pada Audio

CODE

```
# 3. Framing dan Windowing


# Parameter framing
frame_length = 2048         # Jumlah sampel per frame
hop_length = 512            # Jumlah sampel antar frame (overlap)
frame_duration = frame_length / sr
hop_duration = hop_length / sr
overlap = frame_length - hop_length

print(f"Frame Length  : {frame_length} sampel ({frame_duration*1000:.1f}
ms)")
print(f"Hop Length    : {hop_length} sampel ({hop_duration*1000:.1f}
ms)")
print(f"Overlap       : {overlap} sampel ({overlap/sr*1000:.1f} ms)")
print(f"Total Frames  : {1 + (len(y) - frame_length) // hop_length}")
print()

# --- Demonstrasi windowing ---
# Mengambil satu frame
frame = y[0:frame_length]

# Membuat window functions
hann_window = np.hanning(frame_length)
hamming_window = np.hamming(frame_length)
blackman_window = np.blackman(frame_length)

# Visualisasi
fig, axes = plt.subplots(2, 2, figsize=(14, 8))
```

```
# Frame asli
axes[0, 0].plot(frame, color='steelblue', linewidth=0.8)
axes[0, 0].set_title("Frame Asli (Rectangular Window)")
axes[0, 0].set_ylabel("Amplitudo")
axes[0, 0].grid(True, alpha=0.3)

# Window functions
axes[0, 1].plot(hann_window, label='Hann', linewidth=1.5)
axes[0, 1].plot(hamming_window, label='Hamming', linewidth=1.5)
axes[0, 1].plot(blackman_window, label='Blackman', linewidth=1.5)
axes[0, 1].set_title("Window Functions")
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Frame setelah Hann window
windowed_frame = frame * hann_window
axes[1, 0].plot(windowed_frame, color='darkorange', linewidth=0.8)
axes[1, 0].set_title("Frame × Hann Window")
axes[1, 0].set_ylabel("Amplitudo")
axes[1, 0].grid(True, alpha=0.3)

# Perbandingan spektrum
fft_original = np.abs(np.fft.rfft(frame))
fft_windowed = np.abs(np.fft.rfft(windowed_frame))
freqs = np.fft.rfftfreq(frame_length, 1/sr)

axes[1, 1].plot(freqs, 20*np.log10(fft_original + 1e-10),
                alpha=0.7, label='Tanpa Window', linewidth=0.8)
axes[1, 1].plot(freqs, 20*np.log10(fft_windowed + 1e-10),
                alpha=0.7, label='Dengan Hann Window', linewidth=0.8)
axes[1, 1].set_title("Perbandingan Spektrum (dB)")
axes[1, 1].set_xlabel("Frekuensi (Hz)")
axes[1, 1].set_ylabel("Magnitudo (dB)")
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("results/02_framing_windowing.png", dpi=150,
bbox_inches='tight')
plt.show()
```

4. Spektrogram dan Analisis Frekuensi

Untuk mengetahui komponen frekuensi apa saja yang ada di dalam sebuah sinyal, kita harus mengubahnya dari ranah waktu ke ranah frekuensi menggunakan Transformasi Fourier. Karena kita ingin melihat perubahan frekuensi seiring waktu, kita menggunakan **Short-Time Fourier Transform (STFT)**.

STFT menerapkan Transformasi Fourier pada setiap frame yang telah di-*windowing*. Persamaan STFT secara diskrit adalah:

$$X(m,k) = \sum_{n=0}^{N-1} x[n+mH] \cdot w[n] \cdot e^{-j\frac{2\pi kn}{N}}$$

Di mana:

- $X(m,k)$ adalah hasil STFT untuk frame ke-mm dan *bin* frekuensi ke-k.
- H adalah ukuran pergeseran (*hop length*).
- $w[n]$ adalah fungsi jendela.

Spektrogram

Spektrogram adalah representasi visual dua dimensi dari hasil STFT. Sumbu horizontal merepresentasikan waktu, sumbu vertikal merepresentasikan frekuensi, dan warna (intensitas) merepresentasikan magnitudo/amplitudo dari frekuensi tersebut. Karena rentang pendengaran manusia sangat lebar terhadap tekanan suara, magnitudo ini umumnya dikonversi ke dalam skala logaritmik yaitu Desibel (dB):

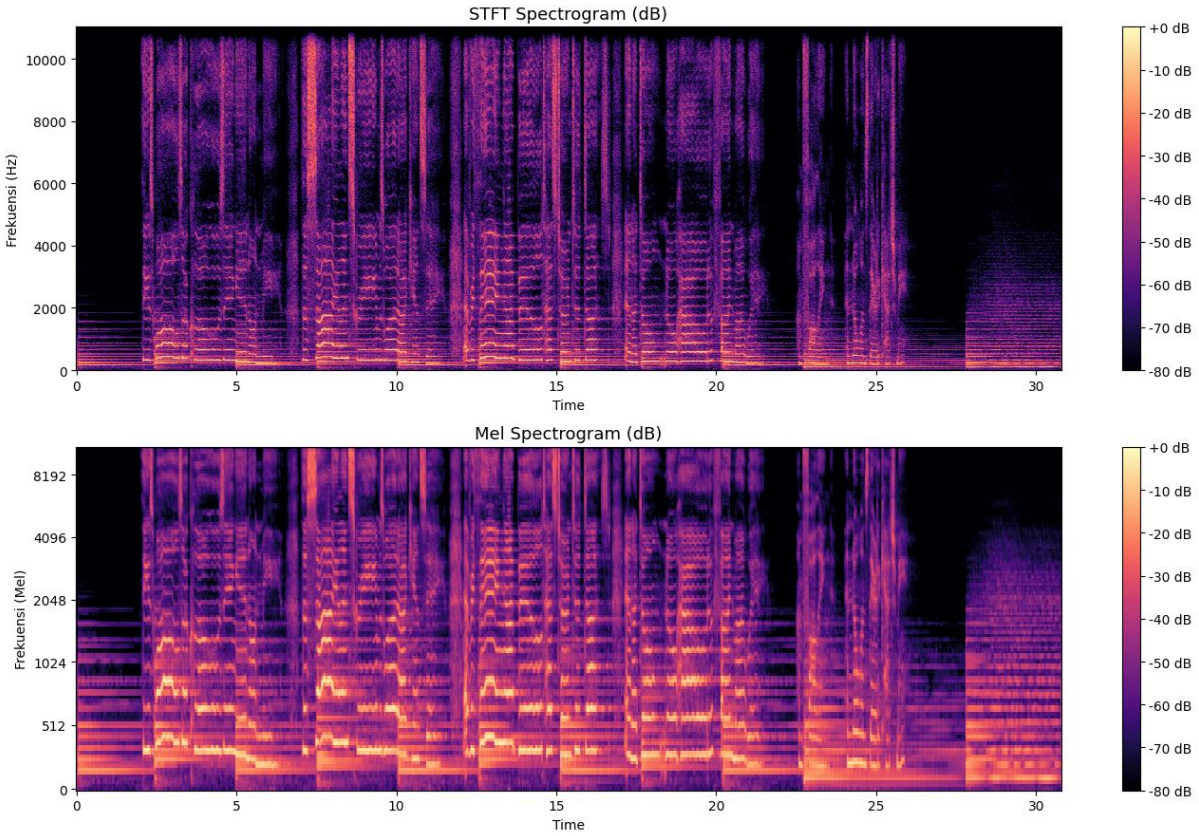
$$S_{dB}(m,k) = 20\log_{10}(|X(m,k)|)$$

Mel-Spectrogram

Pendengaran manusia tidak merespons frekuensi secara linier. Kita lebih sensitif membedakan perubahan frekuensi di nada rendah (misal 100 Hz ke 200 Hz) dibandingkan di nada tinggi (misal 10.000 Hz ke 10.100 Hz). Untuk memodelkan persepsi pendengaran manusia ini, frekuensi dalam Hertz (Hz) dikonversi ke dalam **Skala Mel** menggunakan persamaan:

$$m = 2595\log_{10}\left(1 + \frac{f}{700}\right)$$

Mel-Spectrogram adalah spektrogram biasa yang sumbu frekuensinya telah dipetakan (*warped*) menggunakan *Mel-filterbank*, sehingga visualisasi dan representasi datanya jauh lebih menyerupai cara telinga biologis manusia memproses suara. Fitur ini sangat krusial dan menjadi standar baku (*state-of-the-art*) sebagai input untuk model kecerdasan buatan di bidang pemrosesan audio.



Gambar 2.3: STFT Spectogram dan Mel Spectogram

CODE

```
# 4. Spektrogram (STFT)


# Menghitung STFT
D = librosa.stft(y, n_fft=2048, hop_length=512)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

# Menghitung Mel-Spectrogram
S_mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128,
                                       n_fft=2048, hop_length=512)
S_mel_db = librosa.power_to_db(S_mel, ref=np.max)

fig, axes = plt.subplots(2, 1, figsize=(14, 9))

# STFT Spectrogram
img1 = librosa.display.specshow(S_db, x_axis='time', y_axis='hz',
                                sr=sr, hop_length=512, ax=axes[0],
                                cmap='magma')
axes[0].set_title("STFT Spectrogram (dB)", fontsize=13)
axes[0].set_ylabel("Frekuensi (Hz)")
fig.colorbar(img1, ax=axes[0], format='%+2.0f dB')

# Mel Spectrogram
img2 = librosa.display.specshow(S_mel_db, x_axis='time', y_axis='mel',
                                sr=sr, hop_length=512, ax=axes[1],
                                cmap='magma')
axes[1].set_title("Mel Spectrogram (dB)", fontsize=13)
axes[1].set_ylabel("Frekuensi (Mel)")
fig.colorbar(img2, ax=axes[1], format='%+2.0f dB')

plt.tight_layout()
plt.savefig("results/03_spectrogram.png", dpi=150, bbox_inches='tight')
plt.show()
```

EKSTRAKSI FITUR DOMAIN WAKTU

Fitur domain waktu (*time-domain features*) adalah kumpulan metrik yang diekstraksi langsung dari *waveform* atau gelombang suara mentah tanpa perlu melakukan transformasi sinyal ke domain frekuensi (seperti Transformasi Fourier). Keunggulan utama dari fitur domain waktu adalah kesederhanaan komputasinya yang sangat cepat, sehingga sangat cocok diterapkan pada sistem tertanam (*embedded systems*) atau aplikasi *real-time* yang memiliki keterbatasan daya komputasi.

Meskipun terlihat sederhana, fitur domain waktu mampu memberikan informasi tingkat dasar (*low-level information*) yang sangat penting terkait karakteristik energi, ritme, dan keacakan (*randomness*) dari sebuah sinyal suara. Salah satu fitur domain waktu yang paling fundamental dan luas penggunaannya adalah *Zero Crossing Rate*.

1. ZERO CROSSING RATE (ZCR)

Zero Crossing Rate (ZCR) adalah ukuran yang menghitung seberapa sering sebuah sinyal audio memotong sumbu nol (berubah tanda dari positif ke negatif, atau sebaliknya) dalam suatu periode waktu atau *frame* tertentu.

Secara konseptual, ZCR dapat dianggap sebagai estimasi kasar dari frekuensi dominan atau tingkat "kebisingan" (*noisiness*) dari sebuah sinyal.

- Sinyal yang berfrekuensi tinggi (seperti suara desisan, peluit, atau konsonan frikatif dalam ucapan) akan berfluktuasi dengan sangat cepat, sehingga memotong sumbu nol berkali-kali. Hal ini menghasilkan nilai ZCR yang **tinggi**.
- Sebaliknya, sinyal yang berfrekuensi rendah (seperti suara bass, vokal bernada rendah, atau keheningan murni) berfluktuasi secara perlahan, sehingga jarang memotong sumbu nol. Hal ini menghasilkan nilai ZCR yang **rendah**.

Formulasi Matematis ZCR

Untuk menghitung ZCR pada sebuah *frame* diskrit yang terdiri dari N sampel, kita mengevaluasi perubahan tanda amplitudo antara sampel ke-n dan sampel ke-$(n-1)$. Perubahan tanda ini dapat dideteksi menggunakan fungsi signum (sgn), yang menghasilkan nilai $+1$ untuk bilangan positif dan -1 untuk bilangan negatif (serta 0 jika nilainya tepat nol).

Fungsi signum didefinisikan sebagai:

$$\operatorname{sgn}(x[n]) = \begin{cases} 1 & \text{jika } x[n] > 0 \\ 0 & \text{jika } x[n] = 0 \\ -1 & \text{jika } x[n] < 0 \end{cases}$$

Dengan menggunakan fungsi tersebut, persamaan matematis dari ZCR untuk sebuah *frame* diskrit adalah:

$$ZCR = \frac{1}{2(N-1)} \sum_{n=1}^{N-1} |sgn(x[n]) - sgn(x[n-1])|$$

Di mana:

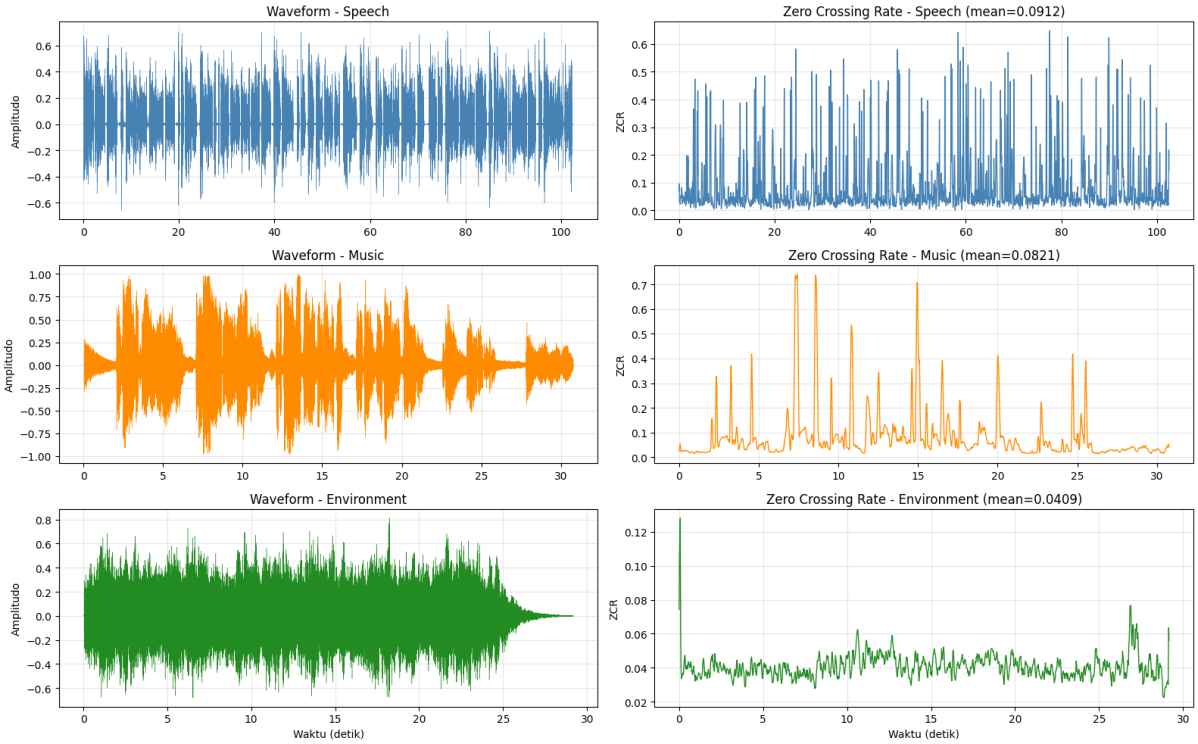
- N adalah total jumlah sampel dalam satu *frame*.
- $x[n]$ adalah amplitudo sinyal pada sampel ke-nn.
- Faktor $\frac{1}{2}$ digunakan karena setiap persilangan nol menghasilkan perbedaan absolut sebesar 2 (misalnya, dari $+1$ ke -1 menghasilkan $|1-(-1)| = 2$). Pembagian dengan 2 mengembalikan nilai ini menjadi 1 per persilangan.
- Pembagian dengan $(N-1)$ digunakan untuk menormalisasi nilai ZCR sehingga hasilnya berada dalam rentang antara 0 (tidak ada persilangan) hingga 1 (persilangan pada setiap sampel berturut-turut).

Signifikansi ZCR Berdasarkan Jenis Audio

ZCR memiliki pola statistik yang khas tergantung pada jenis sumber suara yang dianalisis:

1. **Suara Ucapan (*Speech*):** Suara manusia terdiri dari dua jenis bunyi: bunyi bersuara (*voiced*) seperti huruf vokal (A, I, U, E, O) dan bunyi tak bersuara (*unvoiced*) seperti konsonan frikatif (S, F, Sy, H). Bunyi *voiced* memiliki ZCR yang rendah, sedangkan bunyi *unvoiced* memiliki ZCR yang sangat tinggi dan acak. Oleh karena itu, grafik ZCR pada suara ucapan akan menunjukkan lonjakan-lonjakan ekstrem yang fluktuatif (variansi yang tinggi).
2. **Musik (*Music*):** Musik umumnya terdiri dari perpaduan harmonis berbagai instrumen dan nada yang terstruktur. Meskipun terdapat perkusi yang sesekali meningkatkan ZCR, nilai rata-rata ZCR pada musik cenderung lebih rendah dan lebih stabil (variansi lebih kecil) dibandingkan dengan suara ucapan, mencerminkan periodisitas nada yang konsisten.
3. **Suara Lingkungan (*Environmental / Ambient Noise*):** Suara lingkungan seperti hembusan angin, gemerisik daun, atau derau putih (*white noise*) bersifat stokastik (acak) dengan penyebaran energi frekuensi yang luas namun cenderung tidak memiliki struktur harmonik yang jelas. Hal ini sering kali menghasilkan nilai rata-rata ZCR yang tinggi dan konstan secara temporal (variansi sangat kecil).

Oleh karena perbedaan perilaku ini, ZCR secara historis digunakan sebagai salah satu fitur penentu utama dalam algoritma deteksi aktivitas suara (*Voice Activity Detection* / VAD) serta algoritma pengklasifikasian sederhana untuk membedakan antara suara manusia dan suara musik.



Gambar 3.1: Zero Crossing Rate (ZRC) pada Waveform Speech, Music, dan Environment

CODE

```
# Memuat beberapa jenis audio
y_speech, sr =
librosa.load("/content/Cara_AI_Memahami_Suara_Lewat_MFCC.mp3", sr=22050)
y_music, _    = librosa.load("/content/Echoes_in_the_Dust.mp3", sr=22050)
y_env, _      = librosa.load("/content/Canopy_Whispers.mp3", sr=22050)

# 1. Zero Crossing Rate (ZCR)

# --- Implementasi manual ZCR ---
def compute_zcr_manual(signal, frame_length=2048, hop_length=512):
    """Menghitung ZCR secara manual per frame agar sesuai dengan output
librosa."""
    # Librosa secara default melakukan padding agar frame berada di
tengah (centered)
    # Untuk simplifikasi verifikasi, kita hitung ZCR pada sinyal yang
sama
    frames = librosa.util.frame(signal, frame_length=frame_length,
hop_length=hop_length)

    # Hitung zero crossings untuk setiap kolom (frame)
    zcr_values = []
    for i in range(frames.shape[1]):
        frame = frames[:, i]
        zero_crossings = np.sum(np.abs(np.diff(np.sign(frame))) > 0)
        zcr = zero_crossings / (frame_length - 1)
        zcr_values.append(zcr)

    return np.array(zcr_values)

# Menggunakan librosa
zcr_librosa = librosa.feature.zero_crossing_rate(
    y_speech, frame_length=2048, hop_length=512, center=True
)[0]

# Menggunakan implementasi manual dengan util.frame agar dimensinya sama
zcr_manual = compute_zcr_manual(y_speech)

# Verifikasi kesamaan (menggunakan irisan yang tersedia)
min_len = min(len(zcr_manual), len(zcr_librosa))
print(f"Manual ZCR shape : {zcr_manual.shape}")
print(f"Librosa ZCR shape: {zcr_librosa.shape}")
print(f"Mean Abs Diff    : {np.mean(np.abs(zcr_manual[:min_len] -
zcr_librosa[:min_len])):.6f}")

# --- Perbandingan ZCR untuk berbagai jenis audio ---
zcr_speech = librosa.feature.zero_crossing_rate(y_speech,
frame_length=2048, hop_length=512)[0]
zcr_music  = librosa.feature.zero_crossing_rate(y_music,
frame_length=2048, hop_length=512)[0]
zcr_env    = librosa.feature.zero_crossing_rate(y_env, frame_length=2048,
hop_length=512)[0]

fig, axes = plt.subplots(3, 2, figsize=(16, 10))
audio_data = [
    (y_speech, zcr_speech, "Speech", 'steelblue'),
    (y_music, zcr_music, "Music", 'darkorange'),
    (y_env, zcr_env, "Environment", 'forestgreen')
]
```

```
for i, (y_data, zcr_data, title, color) in enumerate(audio_data):
    # Waveform
    times = np.arange(len(y_data)) / sr
    axes[i, 0].plot(times, y_data, color=color, linewidth=0.3)
    axes[i, 0].set_title(f"Waveform - {title}")
    axes[i, 0].set_ylabel("Amplitudo")
    axes[i, 0].grid(True, alpha=0.3)

    # ZCR
    frames_time = librosa.frames_to_time(np.arange(len(zcr_data)), sr=sr,
hop_length=512)
    axes[i, 1].plot(frames_time, zcr_data, color=color, linewidth=1)
    axes[i, 1].set_title(f"Zero Crossing Rate - {title}
(mean={np.mean(zcr_data):.4f})")
    axes[i, 1].set_ylabel("ZCR")
    axes[i, 1].grid(True, alpha=0.3)

axes[2, 0].set_xlabel("Waktu (detik)")
axes[2, 1].set_xlabel("Waktu (detik)")

plt.tight_layout()
plt.savefig("results/04_zcr.png", dpi=150, bbox_inches='tight')
plt.show()

# Statistik ZCR
print("\n=== Statistik ZCR ===")
print(f"{'Jenis':<12} {'Mean':>8} {'Std':>8} {'Min':>8} {'Max':>8}")
print("-" * 48)
for name, zcr in [("Speech", zcr_speech), ("Music", zcr_music),
("Environment", zcr_env)]:
    print(f"{name:<12} {np.mean(zcr):>8.4f} {np.std(zcr):>8.4f}
{np.min(zcr):>8.4f} {np.max(zcr):>8.4f}")
```

2. ROOT MEAN SQUARE (RMS) ENERGY

Root Mean Square (RMS) *Energy* adalah perhitungan metrik statistik yang digunakan untuk menentukan magnitudo efektif (kekuatan rata-rata) dari sebuah sinyal audio yang berfluktuasi secara dinamis terhadap waktu.

Mengapa kita tidak menggunakan rata-rata amplitudo biasa saja? Karena sinyal audio berfluktuasi antara nilai positif (di atas sumbu nol) dan negatif (di bawah sumbu nol). Jika kita sekadar merata-ratakan seluruh titik sampel pada sebuah gelombang suara, nilai positif dan negatif tersebut akan saling meniadakan (*cancel out*), menghasilkan nilai yang mendekati nol, terlepas dari seberapa keras suara aslinya.

RMS menyelesaikan masalah ini dengan mengkuadratkan setiap sampel terlebih dahulu (sehingga semua nilai menjadi positif), mencari rata-ratanya, lalu mengakarkuadratkan hasilnya untuk mengembalikan skala pengukuran ke satuan aslinya.

Formulasi Matematis RMS Energy

Untuk sebuah *frame* diskrit yang terdiri dari N sampel, nilai RMS dihitung melalui tiga tahapan matematis sesuai namanya (*Root - Mean - Square*):

1. **Square (Kuadrat):** Kuadratkan setiap nilai amplitudo sampel ($x[n]^2$) agar seluruh nilai menjadi positif.
2. **Mean (Rata-rata):** Hitung rata-rata aritmatika dari nilai-nilai kuadrat tersebut ($\frac{1}{N}\sum x$).
3. **Root (Akar Kuadrat):** Tarik akar kuadrat dari rata-rata tersebut (√).

Secara formal, persamaan matematis RMS untuk sebuah *frame* adalah:

$$RMS = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1}(x[n])^2}$$

Di mana:

- RMS adalah nilai *Root Mean Square Energy* dari *frame* tersebut.
- N adalah jumlah total sampel dalam satu *frame*.
- $x[n]$ adalah amplitudo sinyal diskrit pada indeks ke-nn.

Representasi Linear vs. Logaritmik (Desibel/dB)

Nilai yang dihasilkan dari persamaan RMS di atas merupakan representasi **skala linear**. Namun, dalam akustik dan psikoakustik, pendengaran manusia tidak merespons perubahan intensitas suara secara linear. Telinga kita bekerja secara logaritmik—artinya, untuk membuat sebuah suara terdengar dua kali lebih keras, energi fisiknya harus ditingkatkan jauh lebih dari dua kali lipat.

Oleh karena itu, dalam banyak aplikasi analisis audio, nilai RMS linear sering kali dikonversi ke dalam **skala logaritmik**, yaitu Desibel (dB). Konversi ini memberikan representasi perseptual yang jauh lebih sesuai dengan cara manusia mendengar.

Persamaan konversi dari amplitudo linear ke Desibel adalah:

$$RMS_{dB} = 20 \log_{10} \left(\frac{RMS}{RMS_{ref}} \right)$$

Di mana:

- $\mathrm{RMS}_{\mathrm{dB}}$ adalah energi RMS dalam satuan Desibel.
- RMS adalah nilai RMS linear dari *frame* saat ini.
- RMS_{ref} adalah nilai referensi (biasanya nilai amplitudo maksimum absolut dari seluruh sinyal) yang digunakan sebagai titik patokan 0 dB.

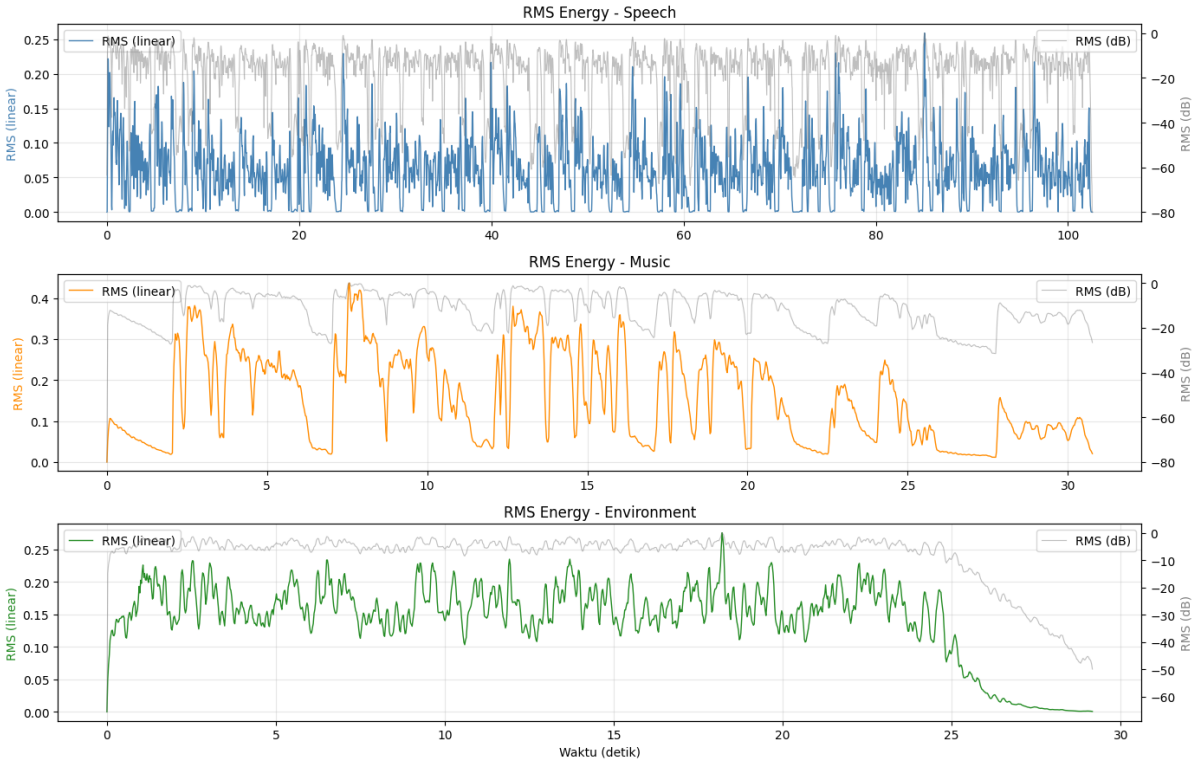
Signifikansi RMS Energy Berdasarkan Jenis Audio

Perbedaan profil energi RMS sangat membantu dalam mengklasifikasikan atau menyegmentasi jenis audio:

1. **Suara Ucapan (*Speech*):** Percakapan manusia dicirikan oleh amplop energi (*energy envelope*) yang sangat dinamis. Akan terlihat bukit-bukit energi tinggi saat suku kata (vokal) diucapkan, diselingi oleh lembah-lembah energi rendah (mendekati nol) saat terjadi jeda napas, spasi antar kata, atau saat mengucapkan konsonan tak bersuara. Fluktuasi RMS pada ucapan sangat tajam dan terstruktur.

2. **Musik (*Music*):** Musik umumnya dikompresi (*mastering*) sedemikian rupa sehingga memiliki rentang dinamis yang lebih padat. Energi RMS pada musik cenderung lebih tinggi secara keseluruhan dan lebih berkelanjutan (*sustained*) karena adanya instrumen pengiring yang terus berbunyi, meskipun tetap memperlihatkan fluktuasi ritmis (seperti ketukan drum atau *bassline*).

3. **Suara Lingkungan (*Environment*):** Suara lingkungan sering kali memiliki profil energi yang relatif konstan (datar) seiring waktu tanpa adanya puncak (*peaks*) atau lembah (*troughs*) yang terstruktur jelas. Energi derau latar (*background noise*) biasanya berada pada level RMS linear yang rendah namun stabil, yang akan terlihat lebih jelas fluktuasinya ketika dikonversi ke skala dB.



Gambar 3.2: RMS Energy pada Speech, Music, dan Enviroment

CODE

```
# 2. Root Mean Square (RMS) Energy


# --- Implementasi manual RMS ---
def compute_rms_manual(signal, frame_length=2048, hop_length=512):
    """Menghitung RMS secara manual per frame."""
    rms_values = []
    num_frames = 1 + (len(signal) - frame_length) // hop_length

    for i in range(num_frames):
        start = i * hop_length
        end = start + frame_length
        frame = signal[start:end]
        rms = np.sqrt(np.mean(frame ** 2))
        rms_values.append(rms)

    return np.array(rms_values)

# Menggunakan librosa
rms_speech = librosa.feature.rms(y=y_speech, frame_length=2048,
hop_length=512)[0]
rms_music  = librosa.feature.rms(y=y_music, frame_length=2048,
hop_length=512)[0]
rms_env    = librosa.feature.rms(y=y_env, frame_length=2048,
hop_length=512)[0]

# Konversi ke dB
rms_speech_db = librosa.amplitude_to_db(rms_speech, ref=np.max)
rms_music_db  = librosa.amplitude_to_db(rms_music, ref=np.max)
rms_env_db    = librosa.amplitude_to_db(rms_env, ref=np.max)
```

```
fig, axes = plt.subplots(3, 1, figsize=(14, 9))
rms_data = [
    (rms_speech, rms_speech_db, "Speech", 'steelblue'),
    (rms_music, rms_music_db, "Music", 'darkorange'),
    (rms_env, rms_env_db, "Environment", 'forestgreen')
]

for i, (rms_lin, rms_db, title, color) in enumerate(rms_data):
    frames_time = librosa.frames_to_time(np.arange(len(rms_lin)), sr=sr,
hop_length=512)

    ax2 = axes[i].twinx()
    axes[i].plot(frames_time, rms_lin, color=color, linewidth=1,
label='RMS (linear)')
    ax2.plot(frames_time, rms_db, color='gray', linewidth=0.8, alpha=0.5,
label='RMS (dB)')

    axes[i].set_title(f"RMS Energy - {title}")
    axes[i].set_ylabel("RMS (linear)", color=color)
    ax2.set_ylabel("RMS (dB)", color='gray')
    axes[i].grid(True, alpha=0.3)
    axes[i].legend(loc='upper left')
    ax2.legend(loc='upper right')

axes[2].set_xlabel("Waktu (detik)")
plt.tight_layout()
plt.savefig("results/05_rms_energy.png", dpi=150, bbox_inches='tight')
plt.show()
```

3. KOMBINASI FITUR DOMAIN WAKTU DAN ANALISIS STATISTIK

Dalam sistem ekstraksi fitur audio, mengandalkan satu jenis fitur saja (misalnya hanya ZCR atau hanya RMS) sering kali tidak cukup untuk mendeskripsikan kompleksitas sebuah sinyal suara. Setiap fitur menangkap dimensi informasi yang berbeda. Oleh karena itu, pendekatan standar dalam pemrosesan audio tingkat dasar adalah menggabungkan beberapa fitur domain waktu secara bersamaan.

Selain fitur *Zero Crossing Rate* (ZCR) yang merepresentasikan frekuensi/keacakan dan *RMS Energy* yang merepresentasikan kekuatan rata-rata sinyal, satu metrik tambahan yang sering digunakan untuk melengkapi analisis domain waktu adalah **Amplitude Envelope**.

1. Amplitude Envelope (AE)

Amplitude Envelope (Selubung Amplitudo) adalah fitur yang mengekstrak nilai puncak (amplitudo maksimum absolut) di dalam setiap *frame*.

Berbeda dengan RMS yang merata-ratakan energi di seluruh *frame*, *Amplitude Envelope* secara spesifik bertugas untuk mendeteksi *transients*—yaitu lonjakan energi ekstrem yang terjadi secara tiba-tiba dan berlangsung sangat singkat. Fitur ini sangat

krusial untuk mendeteksi suara dengan serangan (*attack*) yang tajam, seperti ketukan drum (*snare* / *kick*), ketukan pintu, kaca pecah, atau konsonan plosif pada ucapan manusia (seperti huruf P, T, atau K).

Secara matematis, untuk sebuah *frame* diskrit yang terdiri dari NN sampel, nilai *Amplitude Envelope* dihitung dengan mencari nilai mutlak terbesar:

$$AE = \max_{0 \le n \le N-1} |x[n]|$$

Di mana:

- AE adalah nilai *Amplitude Envelope* untuk *frame* tersebut.
- N adalah jumlah sampel dalam satu *frame*.
- $x[n]$ adalah amplitudo sinyal diskrit pada indeks ke-n.
- max beroperasi untuk mencari nilai tertinggi dari sekumpulan data absolut.

2. Agregasi Statistik Global (*Statistical Pooling*)

Ekstraksi fitur seperti ZCR, RMS, dan AE dilakukan pada tingkat *frame* (lokal), yang berarti sebuah file audio berdurasi beberapa detik akan menghasilkan ratusan atau ribuan nilai fitur yang membentuk sebuah deret waktu (*time-series*).

Namun, sebagian besar algoritma pembelajaran mesin (*machine learning*) tradisional membutuhkan vektor input dengan dimensi yang tetap (*fixed-length vector*), terlepas dari apakah durasi audio tersebut 1 detik atau 10 detik. Untuk menjembatani hal ini, deret waktu dari setiap fitur tingkat *frame* tersebut harus diringkas menjadi nilai tunggal yang merepresentasikan karakteristik keseluruhan dari klip audio tersebut. Proses ini disebut sebagai **Agregasi Statistik Global** atau *Statistical Pooling*.

Metrik statistik yang umum dihitung untuk meringkas distribusi nilai fitur di sepanjang waktu meliputi:

- **Rata-rata (*Mean*):** Menunjukkan nilai pusat atau kecenderungan umum dari fitur tersebut di sepanjang audio.

$$\mu = \frac{1}{K} \sum_{k=1}^{K} F_k$$

- **Standar Deviasi (*Standard Deviation*):** Menunjukkan variansi atau seberapa dinamis perubahan fitur tersebut. Nilai standar deviasi yang tinggi menunjukkan bahwa audio memiliki fluktuasi yang ekstrem (seperti ucapan),

sedangkan nilai yang rendah menunjukkan sinyal yang stabil/datar (seperti derau lingkungan).

$$\sigma = \sqrt{\frac{1}{K}\sum_{k=1}^{K}(F_k - \mu)^2}$$

- **Nilai Maksimum (*Maximum*) dan Minimum (*Minimum*):** Menangkap batas atas dan batas bawah absolut dari fitur, yang berguna untuk mengetahui rentang dinamis sinyal.
- **Nilai Tengah (*Median*):** Menggambarkan nilai sentral yang lebih tahan terhadap data pencilan (*outliers*) dibandingkan rata-rata.

Di mana F_k adalah nilai fitur (misalnya ZCR, RMS, atau AE) pada *frame* ke-k, dan K adalah total keseluruhan *frame* dalam satu klip audio.

Metrik Statistik	**Deskripsi Interpretasi pada Sinyal Audio**
Mean	Kekuatan atau tingkat dasar keseluruhan audio secara umum
Standard Deviation	Tingkat kedinamisan fluktuasi (kontras antara bagian keras dan sunyi)
Maximum	Deteksi puncak tertinggi (*peak*) atau bagian paling intens dari audio
Minimum	Deteksi tingkat *noise floor* (derau dasar) atau keheningan absolut

Dengan menggabungkan berbagai fitur (ZCR, RMS, AE) dan mengekstraksi metrik statistiknya, kita dapat menyusun sebuah **vektor fitur** yang ringkas, berdimensi tetap, namun sangat padat informasi. Vektor inilah yang nantinya diumpankan sebagai data latih ke dalam model kecerdasan buatan untuk tugas klasifikasi suara.

CODE

```
# 3. Kombinasi Fitur Domain Waktu


def extract_time_domain_features(y, sr, frame_length=2048,
hop_length=512):
    """Mengekstraksi semua fitur domain waktu."""
    features = {}

    # ZCR
    zcr = librosa.feature.zero_crossing_rate(y,
frame_length=frame_length,
                                             hop_length=hop_length)[0]
    features['zcr'] = zcr

    # RMS Energy
    rms = librosa.feature.rms(y=y, frame_length=frame_length,
                              hop_length=hop_length)[0]
    features['rms'] = rms

    # Amplitude Envelope
    frames = librosa.util.frame(y, frame_length=frame_length,
hop_length=hop_length)
    ae = np.max(np.abs(frames), axis=0)
    features['amplitude_envelope'] = ae

    # Statistik global
    stats = {}
    for name, feat in features.items():
        stats[f'{name}_mean'] = np.mean(feat)
        stats[f'{name}_std'] = np.std(feat)
        stats[f'{name}_max'] = np.max(feat)
        stats[f'{name}_min'] = np.min(feat)
        stats[f'{name}_median'] = np.median(feat)

    return features, stats

# Ekstraksi fitur
features_speech, stats_speech = extract_time_domain_features(y_speech,
sr)
features_music, stats_music = extract_time_domain_features(y_music, sr)
features_env, stats_env = extract_time_domain_features(y_env, sr)

# Tampilkan statistik dalam tabel
print("\n=== Statistik Fitur Domain Waktu ===\n")
print(f"{'Feature':<25} {'Speech':>10} {'Music':>10}
{'Environment':>12}")
print("=" * 60)
for key in stats_speech.keys():
    print(f"{key:<25} {stats_speech[key]:>10.4f}
{stats_music[key]:>10.4f} {stats_env[key]:>12.4f}")
```

EKSTRAKSI FITUR DOMAIN FREKUENSI

Berbeda dengan fitur domain waktu yang menganalisis bentuk gelombang secara langsung, fitur domain frekuensi (*frequency-domain features*) mengekstrak informasi setelah sinyal ditransformasikan menggunakan *Short-Time Fourier Transform* (STFT). Transformasi ini menghasilkan spektrum magnitudo yang menunjukkan seberapa besar energi yang terkandung pada masing-masing pita frekuensi penyusun suara tersebut.

1. SPECTRAL FEATURES

Karakteristik penyebaran energi spektral ini sangat erat kaitannya dengan persepsi manusia terhadap warna suara atau *timbre*. Empat metrik fundamental yang sering digunakan untuk mendeskripsikan bentuk spektrum ini adalah *Spectral Centroid*, *Spectral Bandwidth*, *Spectral Rolloff*, dan *Spectral Flatness*.

1. Spectral Centroid

Spectral Centroid (Pusat Massa Spektral) adalah metrik yang mengukur di mana letak "titik berat" dari energi frekuensi sebuah suara. Fitur ini sangat berkorelasi dengan persepsi manusia terhadap "kecerahan" (*brightness*) suara.

Jika sebuah suara didominasi oleh frekuensi tinggi (seperti suara simbal, peluit, atau konsonan frikatif), nilai centroidnya akan tinggi dan suara tersebut terdengar "terang" atau "tajam". Sebaliknya, jika suara didominasi oleh frekuensi rendah (seperti suara bass atau vokal berat), nilai centroidnya akan rendah dan suara terdengar "gelap" atau "dalam".

Secara matematis, *Spectral Centroid* dihitung sebagai rata-rata tertimbang (*weighted average*) dari frekuensi yang ada, di mana bobotnya adalah magnitudo spektral pada frekuensi tersebut:

$$C_t = \frac{\sum_{k=1}^{N} f_k \cdot M_t[k]}{\sum_{k=1}^{N} M_t[k]}$$

Di mana:

- Ct adalah *Spectral Centroid* pada frame ke-t.
- fk adalah nilai frekuensi (dalam Hz) pada bin frekuensi ke-k.
- $M_t[k]$ adalah magnitudo spektral pada bin ke-k untuk frame ke-t.
- N adalah jumlah total bin frekuensi.

2. Spectral Bandwidth

Spectral Bandwidth (Lebar Pita Spektral) mengukur seberapa lebar atau sempit penyebaran energi frekuensi di sekitar *Spectral Centroid*-nya. Metrik ini mendeskripsikan "ketebalan" atau "keberagaman" komponen frekuensi dalam sebuah suara.

Sebuah nada murni (seperti hasil garpu tala) memiliki energi yang terkonsentrasi pada satu frekuensi tunggal, sehingga nilai bandwidth-nya sangat rendah (sempit). Sebaliknya, suara derau putih (*white noise*) atau tepuk tangan memiliki energi yang tersebar merata di berbagai frekuensi, sehingga nilai bandwidth-nya sangat tinggi (lebar).

Persamaan matematis untuk *Spectral Bandwidth* (orde kedua) adalah akar dari variansi spektral:

$$BW_t = \sqrt{\frac{\sum_{k=1}^{N}(f_k - C_t)^2 \cdot M_t[k]}{\sum_{k=1}^{N} M_t[k]}}$$

Di mana BW_t adalah *Spectral Bandwidth*, dan parameter lainnya sama seperti pada persamaan *Centroid*.

3. Spectral Rolloff

Spectral Rolloff mengukur batas frekuensi di bawah mana persentase tertentu dari total energi spektral terkonsentrasi. Persentase yang paling umum digunakan adalah 85% atau 95%.

Fitur ini sangat berguna untuk membedakan antara suara harmonik (yang energinya terkumpul di frekuensi dasar dan beberapa overtone pertamanya) dan suara *noisy* (yang energinya terus menyebar hingga ke frekuensi tinggi). *Rolloff* memberikan gambaran tentang batas atas frekuensi efektif dari sebuah sinyal sebelum energinya turun drastis (*roll off*).

Secara matematis, *Spectral Rolloff* adalah frekuensi RtRt sedemikian rupa sehingga memenuhi persamaan:

$$\sum_{k=1}^{R_t} M_t[k] = P \cdot \sum_{k=1}^{N} M_t[k]$$

Di mana:

- R_t adalah frekuensi *Rolloff* (indeks bin batas).
- P adalah persentase ambang batas (misalnya 0.85 atau 0.95).
- $M_t[k]$ adalah magnitudo spektral.

4. Spectral Flatness

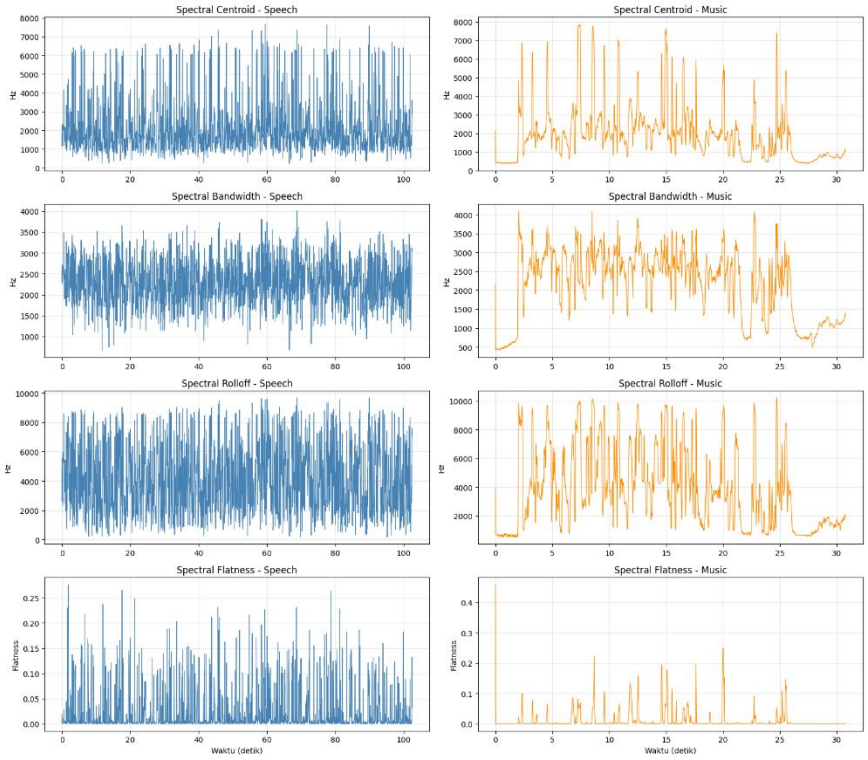
Spectral Flatness (Kemerataan Spektral) adalah metrik yang menilai seberapa "datar" atau merata distribusi energi melintasi semua frekuensi. Metrik ini juga sering disebut sebagai rasio *tonality-to-noise* (nada terhadap derau).

- Nilai *Flatness* yang mendekati **1** berarti spektrumnya sangat datar (energi merata di semua frekuensi), yang mengindikasikan bahwa suara tersebut sangat mirip dengan derau (*noise-like*), seperti suara desisan atau simbal.
- Nilai *Flatness* yang mendekati **0** berarti spektrumnya memiliki puncak-puncak yang tajam, yang mengindikasikan bahwa suara tersebut bersifat tonal (*tone-like*) atau harmonik, seperti suara vokal bernyanyi atau instrumen melodis.

Secara matematis, *Spectral Flatness* didefinisikan sebagai rasio antara rata-rata geometrik (*geometric mean*) dan rata-rata aritmatika (*arithmetic mean*) dari spektrum daya (*power spectrum*):

$$SF_t = \frac{\left(\prod_{k=1}^{N} M_t[k]\right)^{\frac{1}{N}}}{\frac{1}{N}\sum_{k=1}^{N} M_t[k]}$$

Di mana SFtSFt adalah *Spectral Flatness* untuk frame ke-t. Karena rata-rata geometrik selalu lebih kecil atau sama dengan rata-rata aritmatika, rentang nilai SF_t selalu berada di antara 0 hingga 1.



Gambar 4.1: Hasil ekstraksi fitur audio dengan metode Spectral Features

CODE

```
# 1. Spectral Centroid

spec_centroid_speech = librosa.feature.spectral_centroid(
    y=y_speech, sr=sr, n_fft=2048, hop_length=512
)[0]

spec_centroid_music = librosa.feature.spectral_centroid(
    y=y_music, sr=sr, n_fft=2048, hop_length=512
)[0]


# 2. Spectral Bandwidth

spec_bw_speech = librosa.feature.spectral_bandwidth(
    y=y_speech, sr=sr, n_fft=2048, hop_length=512
)[0]

spec_bw_music = librosa.feature.spectral_bandwidth(
    y=y_music, sr=sr, n_fft=2048, hop_length=512
)[0]


# 3. Spectral Rolloff

spec_rolloff_speech = librosa.feature.spectral_rolloff(
    y=y_speech, sr=sr, n_fft=2048, hop_length=512, roll_percent=0.85
)[0]

spec_rolloff_music = librosa.feature.spectral_rolloff(
    y=y_music, sr=sr, n_fft=2048, hop_length=512, roll_percent=0.85
)[0]


# 4. Spectral Flatness

spec_flat_speech = librosa.feature.spectral_flatness(
    y=y_speech, n_fft=2048, hop_length=512
)[0]

spec_flat_music = librosa.feature.spectral_flatness(
    y=y_music, n_fft=2048, hop_length=512
)[0]


# Visualisasi Lengkap

fig, axes = plt.subplots(4, 2, figsize=(16, 14))

titles = ['Spectral Centroid', 'Spectral Bandwidth', 'Spectral Rolloff',
'Spectral Flatness']
speech_feats = [spec_centroid_speech, spec_bw_speech,
spec_rolloff_speech, spec_flat_speech]
music_feats = [spec_centroid_music, spec_bw_music, spec_rolloff_music,
spec_flat_music]
ylabels = ['Hz', 'Hz', 'Hz', 'Flatness']

for i in range(4):
    # Speech
    frames_s = librosa.frames_to_time(np.arange(len(speech_feats[i])),
sr=sr, hop_length=512)
```

```
    axes[i, 0].plot(frames_s, speech_feats[i], color='steelblue',
linewidth=0.8)
    axes[i, 0].set_title(f"{titles[i]} - Speech")
    axes[i, 0].set_ylabel(ylabels[i])
    axes[i, 0].grid(True, alpha=0.3)

    # Music
    frames_m = librosa.frames_to_time(np.arange(len(music_feats[i])),
sr=sr, hop_length=512)
    axes[i, 1].plot(frames_m, music_feats[i], color='darkorange',
linewidth=0.8)
    axes[i, 1].set_title(f"{titles[i]} - Music")
    axes[i, 1].set_ylabel(ylabels[i])
    axes[i, 1].grid(True, alpha=0.3)

axes[3, 0].set_xlabel("Waktu (detik)")
axes[3, 1].set_xlabel("Waktu (detik)")

plt.tight_layout()
plt.savefig("results/06_spectral_features.png", dpi=150,
bbox_inches='tight')
plt.show()
```

2. SPECTRAL CENTROID DI ATAS SPEKTOGRAM

Dalam analisis audio, kita sering kali dihadapkan pada dua jenis representasi fitur: representasi dua dimensi (2D) seperti spektrogram yang memuat informasi sangat detail, dan metrik satu dimensi (1D) berupa deret waktu (*time-series*) seperti *Spectral Centroid* yang bersifat merangkum (meringkas) data.

Menggabungkan (*overlay*) fitur 1D di atas representasi 2D bukan sekadar teknik visualisasi, melainkan sebuah metode analitis untuk memvalidasi bagaimana sebuah metrik matematis secara akurat melacak pergerakan distribusi energi suara yang sesungguhnya.

1. Representasi Energi Logaritmik: Spektrogram Mel

Spektrogram memberikan peta panas (*heat map*) dari distribusi energi suara melintasi waktu (sumbu-X) dan frekuensi (sumbu-Y). Namun, pada analisis tingkat lanjut, sumbu frekuensi linear (Hertz) sering kali dikompresi ke dalam skala perseptual yang disebut **Skala Mel** (*Mel Scale*).

Transformasi ini dilakukan karena telinga manusia bekerja secara logaritmik—kita lebih mudah membedakan perubahan nada di frekuensi rendah (misalnya perbedaan antara nada dasar bas) dibandingkan di frekuensi sangat tinggi (seperti desisan simbal).

Persamaan matematis untuk memetakan frekuensi f dalam satuan Hertz ke dalam skala m (Mel) adalah:

$$m = 2595 \log_{10}\left(1 + \frac{f}{700}\right)$$

Pada *Mel-Spectrogram*, area yang berwarna terang (seperti kuning atau oranye) menunjukkan konsentrasi energi amplitudo yang tinggi pada waktu dan pita frekuensi tertentu, sedangkan area yang gelap (seperti hitam atau ungu) menunjukkan ketiadaan energi atau keheningan.

2. Trajektori Pusat Massa Spektral

Spectral Centroid yang dihitung dari setiap *frame* (potongan waktu) akan membentuk sebuah kurva trajektori yang bergerak naik-turun seiring berjalannya audio. Seperti yang telah dibahas sebelumnya, metrik ini bertindak sebagai "titik berat" atau pusat massa dari spektrum suara.

Persamaan pusat massa untuk *frame* ke-t adalah:

$$C_t = \frac{\sum_{k=1}^{N} f_k \cdot M_t[k]}{\sum_{k=1}^{N} M_t[k]}$$

Di mana f_k adalah frekuensi dan $M_t[k]$ adalah magnitudo energinya.

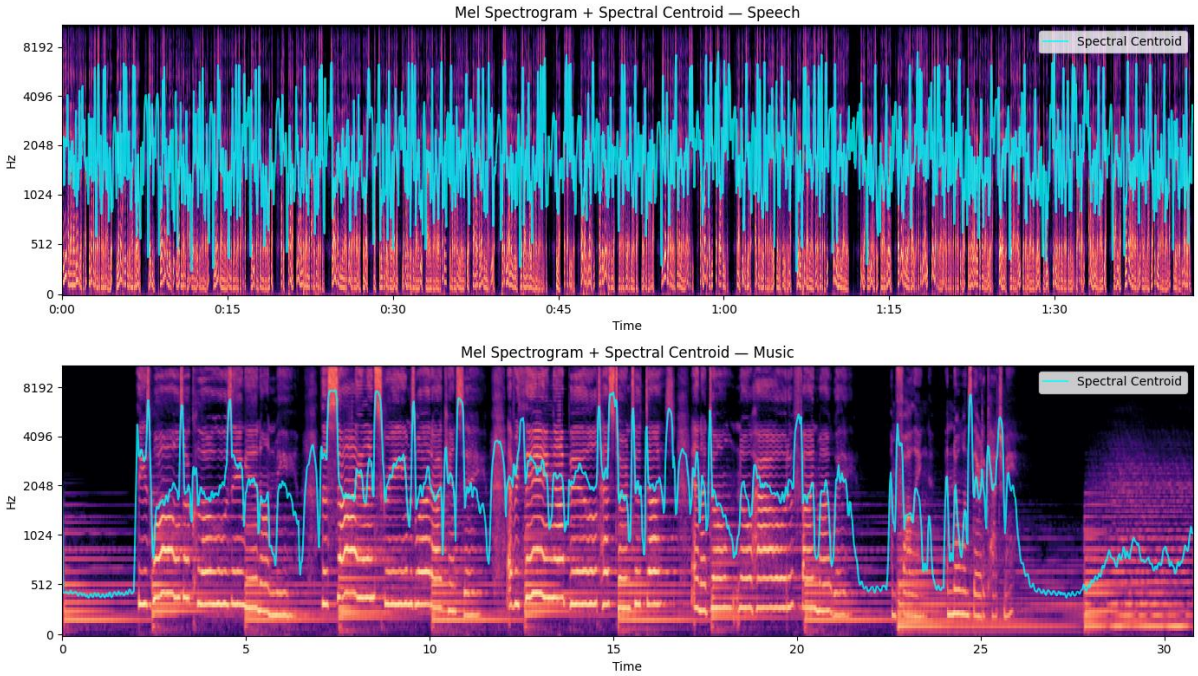
Ketika garis trajektori CtCt ini ditumpangkan (*overlay*) persis di atas peta panas *Mel-Spectrogram*, garis tersebut akan secara otomatis memosisikan dirinya di sekitar area dengan intensitas warna paling terang (energi tertinggi) secara rata-rata:

- **Frekuensi Rendah Dominan:** Jika sebuah suara didominasi oleh dentuman drum bas atau vokal berat pria, area terang pada spektrogram akan menumpuk di bagian bawah sumbu-Y. Akibatnya, garis *Spectral Centroid* akan tertarik turun mendekati sumbu waktu.
- **Frekuensi Tinggi Dominan:** Jika muncul bunyi konsonan berdesis (huruf 'S', 'F', 'Sy') pada ucapan manusia atau ketukan simbal pada musik, area terang akan menyebar ke bagian atas spektrogram. Garis *Spectral Centroid* akan langsung melonjak naik secara tajam.

3. Interpretasi Berdasarkan Jenis Sumber Suara

Melalui penumpangan metrik ini, karakteristik unik dari berbagai jenis suara menjadi sangat mudah dikenali:

- **Pada Suara Ucapan (*Speech*):** Garis *Spectral Centroid* akan terlihat sangat fluktuatif, patah-patah, dan dinamis. Akan terlihat lonjakan garis yang ekstrem ke atas setiap kali seseorang mengucapkan konsonan tak bersuara (karena energi tingginya menyebar luas seperti derau), dan garis akan turun kembali ke area bawah saat mengucapkan huruf vokal (karena energinya terkumpul kuat di frekuensi dasar dan formant rendah).
- **Pada Suara Musik (*Music*):** Pergerakan garis cenderung lebih terstruktur, ritmis, dan berkelanjutan. Meskipun ada fluktuasi dari instrumen perkusi, garis *Spectral Centroid* pada musik umumnya menjaga rata-rata posisi yang lebih konstan karena perpaduan harmonis dari berbagai instrumen yang terus berbunyi mengisi seluruh spektrum frekuensi secara bersamaan.



Gambar 4.2: Spectral Centroid overlapping Mel Spectogram pada Speech dan Music

CODE

```
# Overlay Spectral Centroid pada Spectrogram


fig, axes = plt.subplots(2, 1, figsize=(14, 8))

for idx, (y_data, label, centroid) in enumerate([
    (y_speech, "Speech", spec_centroid_speech),
    (y_music, "Music", spec_centroid_music)
]):
    S = librosa.feature.melspectrogram(y=y_data, sr=sr, n_mels=128)
    S_db = librosa.power_to_db(S, ref=np.max)

    librosa.display.specshow(S_db, x_axis='time', y_axis='mel',
                             sr=sr, hop_length=512, ax=axes[idx],
cmap='magma')

    # Overlay centroid (perlu normalisasi ke skala mel)
    frames_t = librosa.frames_to_time(np.arange(len(centroid)), sr=sr,
hop_length=512)
    axes[idx].plot(frames_t, centroid, color='cyan', linewidth=1.5,
                   label='Spectral Centroid', alpha=0.8)
    axes[idx].set_title(f"Mel Spectrogram + Spectral Centroid — {label}")
    axes[idx].legend(loc='upper right')

plt.tight_layout()
plt.savefig("results/07_centroid_overlay.png", dpi=150,
bbox_inches='tight')
plt.show()
```

3. IMPLEMENTASI MANUAL SPECTRAL CENTROID

Untuk mengekstrak fitur pada domain frekuensi seperti *Spectral Centroid*, tahapan pertama yang harus dilalui oleh setiap sistem pemrosesan audio adalah mengubah sinyal digital mentah (*raw waveform*) yang berada dalam ranah waktu menjadi representasi spektral di ranah frekuensi. Proses ini secara matematis dimungkinkan melalui Transformasi Fourier, yang bertugas memecah gelombang kompleks menjadi komponen-komponen gelombang sinus penyusunnya.

1. Transformasi Sinyal (STFT)

Karena sinyal audio bersifat dinamis dan berubah-ubah (*non-stationary*), kita tidak bisa menerapkan Transformasi Fourier pada keseluruhan file audio sekaligus. Alih-alih, sinyal harus dipecah menjadi bingkai-bingkai pendek (umumnya berdurasi puluhan milidetik) yang saling tumpang tindih (*overlap*). Pada setiap bingkai pendek inilah Transformasi Fourier diterapkan secara mandiri. Teknik ini dikenal sebagai **Short-Time Fourier Transform (STFT)**.

Output dari STFT adalah sebuah matriks bilangan kompleks, di mana nilai absolut dari matriks tersebut merepresentasikan besaran **Magnitudo** (energi) untuk setiap pita frekuensi pada waktu tertentu. Secara formal, magnitudo spektral untuk bingkai waktu ke-t dan *bin* frekuensi ke-k didefinisikan sebagai $M_t[k]$ atau sering disimbolkan dengan $|X_t[k]|$.

2. Pita Frekuensi (Frequency Bins)

STFT menghasilkan kumpulan frekuensi diskrit yang berjarak sama rata (linier), mulai dari 0 Hz (komponen *Direct Current* atau DC) hingga frekuensi maksimal (frekuensi Nyquist, yang besarnya adalah setengah dari *Sampling Rate*).

Setiap indeks k pada hasil STFT tidak merepresentasikan satu angka frekuensi yang pasti, melainkan merepresentasikan sebuah interval atau "keranjang" (*bin*) frekuensi. Nilai frekuensi tengah dari setiap *bin* ke-k (f_k) bergantung pada *Sampling Rate* (f_s) dan resolusi STFT (panjang jendela NFFTNFFT), dan dihitung menggunakan persamaan:

$$f_k = k \cdot \frac{f_s}{N_{FFT}}$$

Di mana k bergerak dari 0 hingga $\frac{N_{FFT}}{2}$. Sebagai contoh, bin ke-0 adalah 0 Hz, dan bin terakhir merepresentasikan frekuensi Nyquist ($\frac{f_s}{2}$).

3. Perhitungan Pusat Massa (Centroid)

Setelah kita memiliki dua komponen utama, yaitu nilai frekuensi setiap *bin* (f_k) dan besaran magnitudo energi pada *bin* tersebut ($M_t[k]$), kita dapat menghitung **Spectral Centroid**.

Spectral Centroid adalah konsep yang dipinjam dari mekanika fisika untuk menghitung titik keseimbangan atau pusat massa suatu benda padat. Dalam konteks audio, benda padat tersebut adalah bentuk spektrum suara, dan kita ingin mencari satu titik frekuensi di mana energi spektral tersebut "seimbang" jika ditimbang secara merata.

Secara konseptual, semakin banyak energi yang terkonsentrasi di frekuensi tinggi (seperti suara desisan tajam), semakin tinggi nilai frekuensi pusat keseimbangannya. Sebaliknya, semakin banyak energi di frekuensi rendah (seperti dentuman *bass*), semakin rendah nilai frekuensi pusat keseimbangannya.

Persamaan matematis untuk menghitung rata-rata tertimbang (*weighted average*) dari pusat massa spektral pada *frame* waktu ke-tt adalah:

$$C_t = \frac{\sum_{k=0}^{K} f_k \cdot M_t[k]}{\sum_{k=0}^{K} M_t[k]}$$

Di mana:

- C_t adalah *Spectral Centroid* (dalam satuan Hertz) untuk *frame* waktu ke-t.
- f_k adalah nilai frekuensi aktual (Hertz) pada *bin* ke-k.
- $M_t[k]$ adalah besaran magnitudo (energi linear) pada *bin* ke-k dan *frame* ke-t.
- K adalah jumlah total *bin* frekuensi ($\frac{N_{FFT}}{2}$).

4. Stabilitas Numerik (Division by Zero)

Dalam praktiknya, terdapat kondisi di mana sebuah *frame* audio mungkin benar-benar sunyi tanpa adanya sinyal sama sekali (energi magnitudo Mt[k]=0Mt [k]=0 untuk semua kk). Jika kondisi ini terjadi, perhitungan penyebut (bagian bawah pecahan) pada persamaan di atas akan bernilai nol.

Secara matematis, pembagian dengan angka nol tidak terdefinisi dan akan menghasilkan galat tak terhingga (*Infinity* atau *Not a Number*). Untuk menjamin stabilitas perhitungan komputasi, sebuah nilai konstanta positif yang sangat kecil (epsilon, disimbolkan $\epsilon\epsilon$, misalnya 10−1010−10) ditambahkan pada penyebut.

Persamaan stabilitas numeriknya menjadi:

$$C_t = \frac{\sum_{k=0}^{K} f_k \cdot M_t[k]}{\sum_{k=0}^{K} M_t[k] + \epsilon}$$

Penambahan ϵ ini sama sekali tidak mengubah nilai akhir *Centroid* secara signifikan saat ada sinyal suara, namun akan menyelamatkan program dari kegagalan sistem saat menghadapi *frame* yang berisi keheningan absolut.

CODE

```
# Implementasi Manual Spectral Centroid


def spectral_centroid_manual(y, sr, n_fft=2048, hop_length=512):
    """Menghitung Spectral Centroid secara manual."""
    # Compute STFT
    D = np.abs(librosa.stft(y, n_fft=n_fft, hop_length=hop_length))

    # Frequency bins
    freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)

    # Spectral Centroid per frame
    centroid = np.sum(freqs[:, np.newaxis] * D, axis=0) / (np.sum(D,
axis=0) + 1e-10)

    return centroid

# Verifikasi
sc_manual = spectral_centroid_manual(y_speech, sr)
sc_librosa = librosa.feature.spectral_centroid(y=y_speech, sr=sr)[0]

print(f"Manual shape : {sc_manual.shape}")
print(f"Librosa shape: {sc_librosa.shape}")
print(f"Max Abs Diff : {np.max(np.abs(sc_manual - sc_librosa)):.6f}")
```